Task 7 — Docker Image Versioning

DEVOPS-007: Dockerfile & Custom Docker Image | Time: 1 Hour

Step 1: Build the first version

```
docker build -t devops-website:v1 .
```

This builds an image from the Dockerfile in the current directory and tags it as devops-website:v1. The tag after the colon marks it as version 1.

Step 2: Modify the website, then build v2

```
docker build -t devops-website:v2 .
```

After changing the website's content/code, rebuilding with a new tag (v2) creates a separate image rather than overwriting v1 — both versions still exist and can be run independently.

Step 3: Build the latest tag

```
docker build -t devops-website:latest .
```

latest is just a tag name like any other — it isn't automatically the newest build. It's a convention many teams use to point at whichever version is the current default, but it has to be built/pushed deliberately to stay accurate.

Step 4: Check the images

```
docker images
```

This lists all local images. I should see three entries for devops-website — v1, v2, and latest — each with its own IMAGE ID (unless latest happens to match one of the others exactly), size, and creation time.

Step 5: Understand the tags

Tag	Meaning
`devops-website:v1`	The first build of the website image
`devops-website:v2`	A newer build, created after modifying the website
`devops-website:latest`	A conventional tag marking the current/default version

Important Question

Why should companies use meaningful image tags instead of always using latest?

Using only latest is risky in production because the tag is mutable — it gets overwritten every time a new image is pushed with that name, so latest today might not be the same image as latest yesterday. This creates several problems: it's impossible to know exactly which code version is actually running,

deployments become unpredictable since pulling "latest" could silently bring in breaking changes, and rolling back to a previous working version becomes difficult or impossible because there's no way to reference what the old "latest" pointed to.

Meaningful tags — version numbers (v1.2.0), git commit hashes, or build/date-based tags — solve this by making every image traceable, reproducible, and reversible. Teams can pin a specific version in deployment configs, know precisely what's running in each environment, and roll back instantly by just redeploying an older, still-existing tag. In short: version tags give control and auditability that latest alone can never provide.